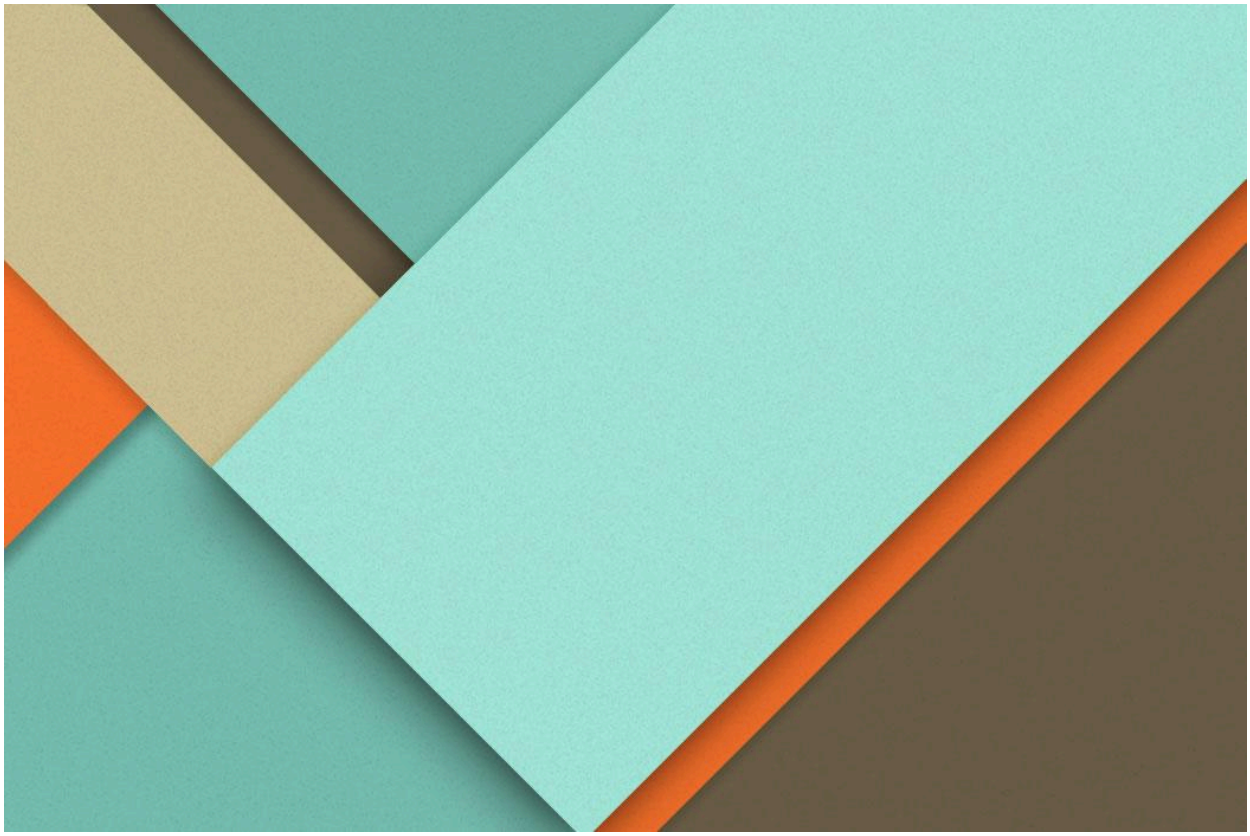

EngBang Programming Language - V2

01 - 26 - 2026

Salek Bin Hossain
Roll: 2107026
Lab group: A1

1. Objectives

- Design a unique programming language syntax (EngBang) using Banglish-style keywords.
- Define strict rules for variable declaration, updates, constants, default initialization, and type casting.
- Define a unified conditional system where all condition operators/operations are written using { } and condition lists use [ ].
- Support Python-like chained comparisons (e.g., 10 <= x <= 20, a == b == c) using EngBang bindings.
- Document corner cases so the language remains unambiguous and parser-friendly.

2. Introduction

EngBang v2 is designed to avoid the typical C/Python look and feel. Instead of placing operators directly inside conditions, EngBang evaluates conditions using a binding-first model: var -> value declares what is being compared, and a unified { } condition token decides how the comparison is performed. Logical composition is handled using lists: a single [...] represents AND across bindings, and multiple lists represent OR. This specification focuses on two core parts: variables and conditional statements.

3. Language Specification: Part 1 — Variables

3.1 Data Types

Type	Meaning	Valid Values (Examples)
shonkha	Integer	-5, 0, 10, 999
doshomik	Float	3.14, -0.50, 10.0
lekha	Text/String	"hello", "Talha \"Safin\""
shotto	Boolean	0 or 1 only

Boolean convention: shotto values are strictly 0 (false) and 1 (true).

3.2 Identifiers (Variable Names)

Rules:

- Must start with A–Z, a–z, or underscore (_).
- May contain A–Z, a–z, 0–9, underscore (_) after the first character.
- Reserved words cannot be used as identifiers.

reserved words list:

```
rakho, bodlao, pakka, khaali, ar, bolo, ->, |, [], {}, [ ], { }
```

Examples:

```
# Valid
10 rakho x -> shonkha |
khaali rakho _temp -> doshomik |
"hi" rakho user_name -> lekha |

# Invalid
10 rakho 1x -> shonkha |       # starts with digit
10 rakho rakho -> shonkha |     # reserved word
```

3.3 Statement Termination

Rule: Every statement must end with |, including statements inside conditional blocks.

```
bolo "Hello" |
10 bodlao x |
```

3.4 Declaration (rakho)

```
value rakho var_name -> type |
```

Rules:

- rakho declares a variable.
- Redeclaration is not allowed (use bodlao to change values).

```
# Valid
10 rakho x -> shonkha |
"Talha" rakho name -> lekha |

# Invalid (redeclaration)
10 rakho x -> shonkha |
20 rakho x -> shonkha |   # ERROR
```

3.5 Multiple Declarations in One Line (comma-free)

Use "**ar"** to chain declarations; end the line with one |.

```
10 rakho x -> shonkha ar 20 rakho y -> shonkha ar "hi" rakho s -> lekha |
```

3.6 Default Values (khaali)

```
khaali rakho var_name -> type |
```

Default mapping:

- khaali -> shonkha = 0
- khaali -> doshomik = 0.0
- khaali -> lekha = ""
- khaali -> shotto = 0

```
khaali rakho count -> shonkha |
khaali rakho avg -> doshomik |
khaali rakho title -> lekha |
khaali rakho flag -> shotto |
```

3.7 Update (bodlao)

```
value bodlao var_name |
```

Rules:

- Variable must be declared before update.
- Type mismatch is an error (strict typing).

```
# Valid
10 rakho x -> shonkha |
20 bodlao x |

# Invalid: undeclared
20 bodlao y |  # ERROR

# Invalid: type mismatch
10 rakho x -> shonkha |
"hi" bodlao x |  # ERROR
```

3.8 Constants (pakka)

```
value pakka rakho NAME -> type |
```

Rules:

- pakka variables cannot be updated with bodlao.
- Constants must be initialized (explicit value or khaali).

```
10 pakka rakho LIMIT -> shonkha |
"v2" pakka rakho VERSION -> lekha |
khaali pakka rakho FLAG -> shotto |

20 bodlao LIMIT |  # ERROR: cannot update pakka
```

3.9 Type Casting (banaw)

```
value banaw type rakho var -> type |
value banaw type bodlao var |
```

```
"123" banaw shonkha rakho n -> shonkha |
"3.50" banaw doshomik rakho f -> doshomik |
123 banaw lekha rakho s -> lekha |
"999" banaw shonkha bodlao n |
```

4. Language Specification: Part 2 — IF / ELSEIF / ELSE

EngBang conditionals use a binding-first model with unified { } condition items and [ ] groups.

4.1 IF / ELSEIF / ELSE Format

```
# IF
<bindings> {
  <groups> {
        <statements>   # each ends with |
  }
}

# ELSEIF (optional, can repeat)
<bindings> {
  <groups> {
        <statements> |
  }
}

# ELSE (optional)
[] {
  <statements> |
}
```

4.2 Bindings

```
var -> value ar var -> value ar ...
```

- Binding order matters: condition items apply in the same order.
- Value may be a literal or an identifier (supports chained comparisons).
- Same variable may appear multiple times in bindings (x -> 10 ar x -> 20).

4.3 AND / OR with [ ] Groups

- One [...] is AND across bindings.
- Multiple [...][...] are OR groups.
- [] is reserved only for ELSE.

```
a -> 10 ar b -> 0 {
  [{>=}, {% 2}][{==}, {}] {
          bolo "OK" |
  }
}
[] { bolo "NO" | }
```

4.4 Strict Count Matching

```
# bindings = 3
a -> 1 ar b -> 2 ar c -> 3 {
  [{==}, {}, {>=}] { bolo "valid" | }
}

# invalid (count mismatch)
a -> 1 ar b -> 2 ar c -> 3 {
  [{==}, {>=}] { bolo "ERROR" | }
}
```

4.5 Condition Item Types

A) Comparator-only: {==} {!=} {>=} {<=} {>} {<}

```
x -> 10 {
  [{>=}] { bolo "x >= 10" | }
}
```

B) Transform then equality: {% 2}, {^ 5}, {+ 10}, {>> 1} ... means (var OP k) == value

```
x -> 0 {
  [{% 2}] { bolo "even" | }
}

x -> 10 {
  [{^ 5}] { bolo "match" | }
}
```

C) Skip: {} ignores the binding in the current AND list.

```
cgpa -> 3.75 ar attendance -> 80 ar hasQuota -> 1 {
  [{>=}, {>=}, {}] { bolo "needs only cgpa+attendance" | }
}
```

4.6 Execution Order

- Evaluate IF first; test OR-groups left to right; first true group runs.
- If IF fails, evaluate ELSEIF blocks in order.
- If none match, run ELSE ([] { ... }) if present.

5. Python-like Chained Comparisons

```
 # 10 <= x <= 20

x -> 10 ar x -> 20 {
  [{>=}, {<=}] { bolo "in range" | }
}
[] { bolo "out of range" | }

# a == b == c == d
a -> b ar b -> c ar c -> d {
  [{==}, {==}, {==}] { bolo "all equal" | }
}
[] { bolo "not equal" | }
```

6. C ↔ EngBang Example

Direct mapping between common C conditions and EngBang bindings + condition items.

Example 1: Even / Odd

C

```
if (x % 2 == 0) printf("jor\n");
else printf("bijor\n");
```

EngBang

```
x -> 0 {
  [{% 2}] { bolo "jor" | }
}
[] { bolo "bijor" | }
```

Example 2: Positive / Zero / Negative

C

```
if (n > 0) printf("positive\n");
else if (n == 0) printf("zero\n");
else printf("negative\n");
```

EngBang

```
n -> 0 { [{>}]  { bolo "positive" | } }
n -> 0 { [{==}] { bolo "zero" | } }
[]         {          bolo "negative" | }
```

Example 3: Scholarship (AND + OR + Skip)

C

```
if ((cgpa>=3.75 && attendance>=80) || hasQuota==1)
    printf("Eligible\n");
else
            printf("Not eligible\n");
```

EngBang

```
cgpa -> 3.75 ar attendance -> 80 ar hasQuota -> 1 {
  [{>=}, {>=}, {}][{}, {}, {==}] { bolo "Eligible" | }
}
[] { bolo "Not eligible" | }
```

Example 4: Grade Range

C

```
if (mark >= 80 && mark <= 100) printf("A+\n");
else printf("Not A+\n");
```

EngBang

```
mark -> 80 ar mark -> 100 {
  [{>=}, {<=}] { bolo "A+" | }
}
[] { bolo "Not A+" | }
```

Example 5: Boolean (shotto as 0/1)

C

```
if (!flag) printf("FALSE branch\n");
else printf("TRUE branch\n");
```

EngBang

```
flag -> 0 { [{==}] { bolo "FALSE branch" | } }
[]        {        bolo "TRUE branch" | }
```